

Process Notes - Madlen Learning Loop

Decisions and deliberate UX

I built the three requested products as one learning lifecycle - lesson preparation before class, guided student support during learning, and essay feedback after learning - because Madlen's value is easier to understand as a connected teacher outcome than as three unrelated AI demos. Each module still works independently.

The interface uses Madlen's official logo and a restrained cream, orange, and sage system to feel calm and familiar to non-technical educators. Navigation, controls, and system states are English because the case and Madlen's public product are English-first. Generated learning content and its result labels switch together between English and Turkish; mismatched-language model responses are rejected. Copy actions reduce rework, progressive hint labels make the chatbot's pedagogy visible, and essay scores remain advisory with a clear teacher-review warning.

AI and technical tools used

- Groq API + **openai/gpt-oss-120b**: live lesson, chat, and essay generation; selected for a zero-cost tier and strict JSON-schema output.
- Codex: implementation, prompt iteration, research synthesis, tests, build checks, deployment support, and browser-based end-to-end QA.
- OpenAI image generation: created the paper-planner illustration for the social post. Exact copy and the official logo were added separately to prevent AI spelling or brand errors.
- Web research: checked Madlen, MagicSchool, Brisk, Groq, and Netlify claims against current sources.
- Next.js, TypeScript, Zod: one responsive application, server-only API calls, bounded inputs, and validation of both user input and model output.

Real changes during development

I replaced a planned Groq SDK with native server-side **fetch** to reduce dependencies. I considered automatic Gemini fallback but kept one provider because a second secret, data path, and response contract would need equivalent validation; the UI instead preserves input and offers retry. Deployment moved from an initial Vercel route to Netlify when Netlify provided the clearer free public path for this repository. Live testing also led to exact essay-quotation checks, bilingual output validation, and a social-post pivot from the broad Learning Loop concept to the brief-compliant PYP Unit Planner.

Remaining limitations and next step

This is a review-scale prototype: there is no authentication, persistence, shared rate-limit store, school administration, or curriculum retrieval layer. Groq's free quota is not classroom-scale capacity, and AI output still requires educator review. Next, test the full loop with 5-10 teachers; if completion, usefulness, and alignment quality meet the pilot thresholds, add cited curriculum sources, saved/exportable work, school controls, and a deliberately tested provider fallback.